CPSC 6820 Literature Component

The +25 graduate requirement for Assignment 4.1. Jeff Branyon. Summer 2026.

Included as the final part of this document rather than as a separate upload, because only one Canvas item was open for Week 4. Everything before this page is Assignment 4.1. This part is self-contained and can be graded on its own.

Paper

Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems* (CHI '24), May 11-16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, Article 142, 16 pages. https://doi.org/10.1145/3613904.3641936

Summary and connection

Methodology. Mozannar et al. ran a 60-minute observational study of 21 programmers using GitHub Copilot in VS Code on a remote VM. Familiarity ranged from 11 who had never used it to 7 weekly users. Each received one of eight mostly-Python tasks in a 20-minute block, delivered as an image so participants could not paste the prompt and had to author their own. Telemetry logged every show, accept, reject, and browse event, segmenting each session. Immediately afterward, participants reviewed their own screen recording segment by segment and labeled each with one of twelve CUPS states (CodeRec User Programming States). That retrospective self-labeling is the methodological core: rather than inferring intent from telemetry, it asks developers what they were doing. Analysis covered 3,137 labels; 353 uncertain ones were discarded, not reinterpreted. There was deliberately no Copilot-free control: the aim was time allocation within AI-assisted work, not speedup.

Findings. Verifying suggestions was the largest single activity at 22.4% of session time, ahead of writing new functionality at 14.1%, a task the tool newly introduced. Copilot-specific states consumed 51.5% of sessions. The transition graph shows that deferring thought on a suggestion leads to verifying it later with probability 0.54, a pattern they name **verification debt**: deference postpones review rather than removing it. Most consequentially, counting verification that happens *after* acceptance raises mean verification time from 3.25 to 15.21 seconds, roughly fivefold. Acceptance-rate metrics therefore understate review cost systematically.

Connection to my lab. Verification debt is the empirical case for my Checkpoint 2, and I did not have that argument when I designed it. My instinct was that two gates would do: plan approval, then a final diff review. The fivefold correction argues otherwise, because one end-of-task review must absorb debt accrued across the entire run, which is exactly what a mid-stream gate prevents. My lab reproduced the cost asymmetry at small scale. The agent produced Part 1 in roughly five minutes of wall clock; verifying four of its claims against the baseline commit, reading a 126-line diff line by line, and auditing 23 agent-

written tests for gaming is the part that does not compress, and it took nearly all of my own effort. Delegation did not remove work, it converted authoring into verifying. That reframes what a checkpoint is worth: not that it prevents a bad merge, but that it divides an unreviewable quantity of verification into amounts a human will actually perform.